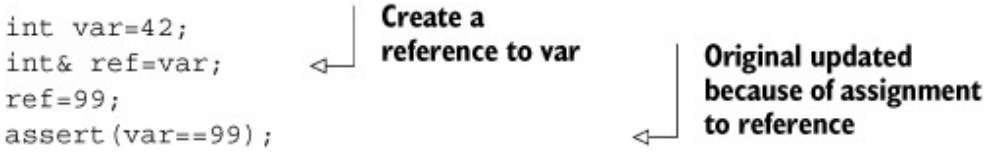
Appendix A. Brief reference for some C++11 language features

The new C++ Standard brings more than just concurrency support; there are a whole host of other language features and new libraries as well. In this appendix I give a brief overview of the new language features that are used in the Thread Library and the rest of the book. Aside from `thread_local` (which is covered in section A.8), none of them are directly related to concurrency, though they are important and/or useful for multithreaded code. I've limited this list to those that are either necessary (such as rvalue references) or serve to make the code simpler or easier to understand. Code that uses these features may be difficult to understand at first because of lack of familiarity, but as you become familiar with them, they should generally make code easier to understand rather than harder. As the use of C++11 becomes more widespread, code making use of these features will become more common.

Without further ado, let's start by looking at *rvalue references*, which are used extensively by the Thread Library to facilitate transfer of ownership (of threads, locks, or whatever) between objects.

A.1. Rvalue references

If you've been doing C++ programming for any time, you'll be familiar with references; C++ references allow you to create a new name for an existing object. All accesses and modifications done through the new reference affect the original, for example:



The references that we've all been using up to now are *lvalue references*—references to lvalues. The term *lvalue* comes from C and refers to things that